PlayerRadar: A Custom Dashboard for Scouting Basketball Players

Fredryk Nyström* ETH Zürich
Dario Tenore† ETH Zürich
Patrik Möcke‡ ETH Zürich
Yanik Künzi§ ETH Zürich

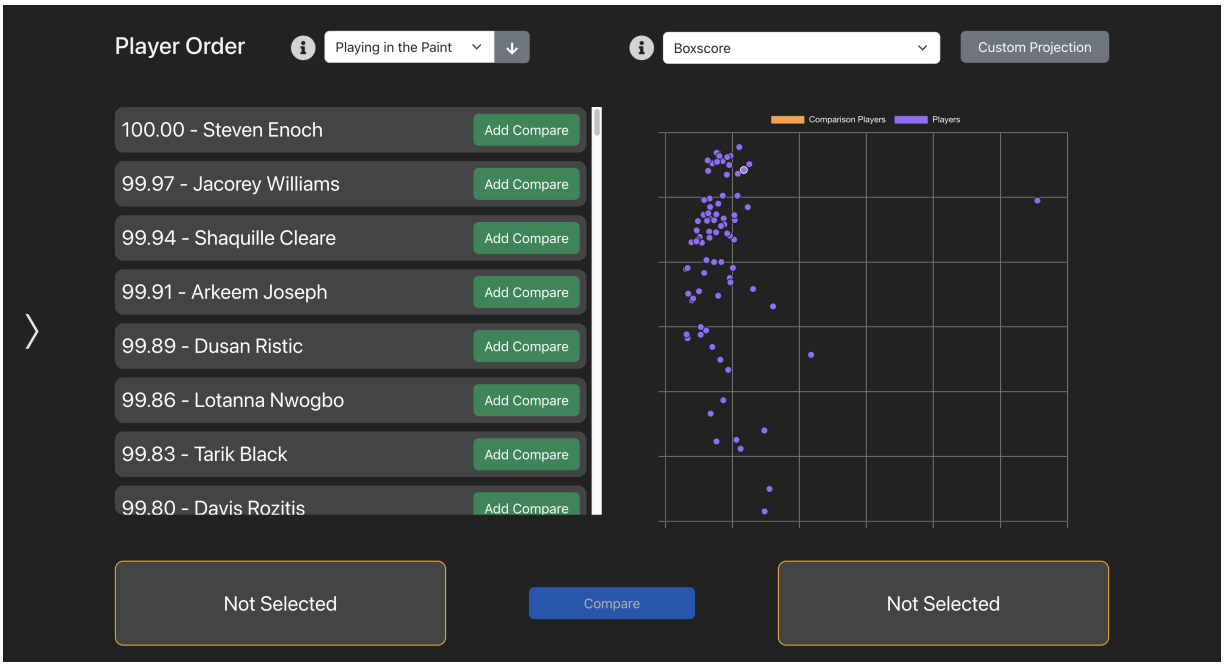

Figure 1: The web interface of our tool. On the left-hand side a list of players is visible which can be compared and inspected for further details. On the right-hand side the output of customizable t-SNE projections is visualized.

Abstract

This project aimed to create an intuitive interface for basketball team scouts to discover new talent by interactively exploring a vast, high-dimensional dataset comprising hundreds of players from various leagues over multiple seasons. This data-driven approach contrasts with traditional scouting techniques and aims to enhance the scouting process by enabling teams to make more informed decisions.

To manage the high-dimensional data, we employed several unsupervised machine learning techniques, specifically, methods based on clustering and projections, assisting scouts in navigating their options. The application of these methods to player statistics resulted in intuitive and easily interpretable visualizations and metrics, significantly simplifying the scouting process.

Index Terms: Interactive and explainable AI, machine learning, web development.

1 Introduction

Basketball is a highly intricate sport on the court, but the complexity extends beyond gameplay to encompass a vast array of statistics and analytics collected off-court. The motivation for gathering such extensive data is underpinned by several key objectives. Primarily, data is collected to enhance team and player performance, facilitate comparisons among players, teams, or leagues, and, perhaps most crucially, to scout new talent. Traditionally, scouting involved physically observing potential players; however, much of the contemporary scouting process now relies heavily on pre-evaluating players through their data.

Our tool addresses this modern approach to scouting. Although analyzing raw data can be effective, it is often tedious and time-consuming. Our solution employs advanced statistical techniques and machine learning (ML) to transform raw player data into a more concise and readable format without compromising its utility. This tool allows scouts to visually compare player statistics in a two-dimensional space, with the flexibility to adjust and customize the ML models used. Consequently, this enables a more intuitive and efficient method for searching and comparing players.

2 Data

The data utilized for our tool was provided by our teaching assistant and encompasses a wide range of European basketball leagues.

2.1 Raw Data

Specifically, we have acquired player data from the past three seasons from leagues such as Liga ACB (Spain), Basketball Bundesliga (Germany), and Euroleague (Europe-wide competition) but also many more. This dataset includes all the commonly collected basketball statistics. Examples of data fields include the percentage of three-pointers attempted, field goals attempted, rebounds, among

*e-mail: fnystroem@student.ethz.ch
†e-mail: dtenore@student.ethz.ch
‡e-mail: pmoecke@student.ethz.ch
§e-mail: ykuenzi@student.ethz.ch

others. Each data point is recorded on a per-game basis; for instance, "minutes played" represents the average minutes played per game in a given season and league. Due to the extensive amount of raw data and the inconsistency in the completeness of some data fields, we undertook data augmentation and cleaning through preprocessing.

2.2 Data Preprocessing

Our data preprocessing pipeline can be roughly divided into the following five stages:

1. Exploratory data analysis and feature selection
2. Treatment of duplicate and missing values
3. Treatment of insufficient data
4. Aggregation of statistics
5. Normalization and standardization

In the first step, we conducted exploratory data analysis and feature selection by plotting histograms for all approximately 120 features and visually inspecting them. This analysis revealed two primary patterns: most features followed a Gaussian-like distribution, while some exhibited an exponential distribution. We eliminated features with an overwhelmingly large number of zeros, as it was unclear whether these zeros represented actual values or missing data. Additionally, we found that features expressed as percentages were redundant due to their clear dependence on other features. For example, we had 'post up attempted', 'post up made', and 'post up, %' where 'post up, %' is derived as 'post up made' divided by 'post up attempted'. Consequently, we removed most of these percentage-based features.

The dataset we received was already very clean, with almost no missing values. However, for the statistic called "effective field goal percentage" (eFG%), some entries were zero despite not being valid. Therefore, we manually computed this statistic using the eFG% formula. Out of our thousands of data points, we found fewer than 20 duplicate entries. A duplicate entry meant having two rows where the player, season, competition, and team were identical. Due to the low number of these duplicates, we simply removed them. It is also important to note that it is possible and valid for a player to participate in multiple competitions (e.g., national league and European league) or for multiple teams (e.g., if traded) within the same season. This issue will be addressed in step 4.

In step 3, we addressed the issue of insufficient data. Many players had only played a few games with minimal minutes, resulting in unreliable statistics that do not accurately reflect their actual performance. For example, a player who played only two games in a season and attempted one shot in each game, making both, would have a 100% shooting accuracy. This statistic is clearly inflated and unsuitable for training machine learning models. After some trial and error, we determined that a player must have played at least 230 minutes in a season for their data to be considered reliable.

We decided to treat each player within a season as a single data point. As previously mentioned, players can appear multiple times within the same season, necessitating the aggregation of their features in step 4. Specifically, if a player played for various teams or in multiple competitions within the same season, we computed a weighted average of these rows. We weighted each row by the total minutes played (games played multiplied by minutes per game). For each player requiring aggregation, we created a new data point containing the weighted average of his statistics and removed the original rows. This approach means that the same player in different seasons is treated as separate data points for the machine learning models. This is a reasonable assumption because players tend to develop or regress from season to season, and we needed more data for our models. All these steps resulted in clean data, which is also used in our database discussed in section 3.1.

Our data contains statistics heavily dependent on a player's time on the court per game. Therefore, we normalized all features, except those already expressed as percentages, by "minutes played". For example, if a player had a game average of 20 minutes played, almost all other features were divided by 20. Additionally, since our machine learning models assume normally distributed variables, we applied a Yeo-Johnson transformation to features that exhibited an exponential-like distribution, as observed in step 1. This transformation is crucial for clustering algorithms, for instance, because, without it, variables with exponential distributions would dominate the algorithm. We verified the transformation by plotting histograms and confirmed that the results were satisfactory, with all features now approximating a Gaussian distribution. Finally, given that our features were on vastly different scales, we applied standardization to ensure that our data approximately followed a standard normal distribution. With these preprocessing steps complete, our data was ready for use in our machine learning models.

3 Backend

The backend of our project can be divided into two distinct components: the database setup and the API that connects the database to the frontend.

3.1 Database Setup

For our project, we chose SQLite due to our familiarity with this database system and its ease of use. Given that our production environment exclusively involves reading from the database, and the number of records and tables is manageable, we did not find it necessary to switch to a more advanced database management system like PostgreSQL.

The data we received was in the form of several CSV files, each containing statistics collected throughout a season in a single league. First, we designed a database schema that aligns with the structure of our data. We created the following tables to store the data:

- The *Players* table stores the name and a unique ID for each player, allowing us to refer to players using their IDs instead of their names, which helps prevent inconsistencies.
- Similarly, the *Team* and *League* tables link each team and league name to a unique ID.
- The *Stats* table stores the actual statistics of the players, with each record in this table associated with a player, league, and specific season.

As described in section 2.2, we found some inconsistencies in the data which we addressed by deleting the affected records. As the statistics stored in the database are intended for presentation to the user rather than for training models, we did not perform any further transformations on them.

3.2 API

To enable the frontend to interact with the data, we provide four RESTful API endpoints:

- The *Player* endpoint returns the names and IDs of all players in the database. Optionally, players can be filtered based on their names.
- The *Overview-Stats* endpoint provides access to the most important statistics of the players. This endpoint was designed to improve application responsiveness, as querying all available statistics for many players was slow and most of the data was not utilized in the overview.

- The *Stats* endpoint returns all available statistics. This detailed data is shown to the user on demand. Since in-depth statistics are displayed for at most two players simultaneously, the amount of data transferred is small, resulting in minimal latency.

- The *Projection* endpoint provides access to the x and y coordinates of players when projected down from the high-dimensional space given by all the statistics of their last season. This feature allows users to easily find players with similar statistics, indicating a similar style of play. A detailed discussion about this can be found in section 5.2.

4 Frontend

Our frontend is designed as an interactive dashboard, prioritizing accessibility and minimizing clutter. Our approach involves displaying only the essential elements, while still providing the option to view additional information when necessary. To elaborate further, we will briefly describe the primary components of the frontend.

4.1 Dashboard

The dashboard can be broadly divided into four sections.

At the top, there are selectors that facilitate ordering the players, selecting their ML projections, and a button that enables the customization of these projections. Additionally, there are two info icons that provide explanations of the selected ordering metric and the ML projection when hovered over.

To the left, there is the player list, which comprises all players that match the current filters. The ordering and selection of players are based on the aforementioned selectors. Each player item consists of three components:

- A value corresponding to the player's statistic relevant to the current ordering.

- The player's name.

- A button that allows the player to be added to or removed from the comparison.

On the right side of the player list, there is a two-dimensional scatter plot that uses the currently selected ML projection to determine the player coordinates. This plot highlights players added to the comparison and also responds when a player item in the player list is hovered over. The primary objective is to utilize spatial ordering to visualize similarities or differences among players that might not be immediately apparent when only considering their statistics.

For both the player list and scatter plot, only the top hundred search results are shown to avoid cluttering the plot. Clicking on a player in either the list or the plot will open their individual stat screen in a pop-up window. This screen follows the same layout as the comparison screen; thus, it will not be detailed here.

The final section of the dashboard is the comparison bar at the bottom. When one or two players are selected, their names appear, and their scatter plot points are highlighted in orange. Furthermore, if two players are selected, clicking on the blue "Compare" button opens the comparison screen.

4.2 Comparison Screen

Whenever the "Compare" button is pressed, the comparison screen appears. It is vertically divided into three sections.

In the header, the names of the two players being compared are displayed.

Beneath the header, there is a radar chart that presents the five custom statistics for each player. Explanations for these five statistics can be viewed by hovering over the "i" icon next to the chart.

At the bottom, there is a tab-based table that displays all available statistics. This table is divided into multiple sections, each showing the corresponding statistics for both players. Higher values are indicated by a light green font and an upward-facing triangle, whereas lower values are displayed in a coral font with a downward-facing triangle. When the statistics are equal, they are shown in white.

As previously mentioned, the stat screen for individual players is identical to the comparison screen, with the exception that the radar chart only displays the values for the single player, and the table only shows the player's individual row of values.

4.3 Filter Screen

The filter screen can be accessed by clicking on the large button on the left, which features a right angle bracket.

Upon opening, the filter window will slide in from the left side of the screen. It contains the following elements, arranged from top to bottom:

- A search field that allows users to search for players by their names.

- A season selector that enables users to specify which season's data is being used.

- A radar chart with two values for each axis: a minimum and a maximum value. By adjusting these values, users can filter players based on their stats falling within the selected range. This feature provides a quick and efficient method for scouting potential ideal players.

4.4 Custom Projections

As previously mentioned, in addition to the seven available ML projections, there is functionality to create custom projections.

This can be initiated by clicking on the "Custom Projection" button located at the top right of the dashboard. Upon clicking, a new window appears containing toggles for each statistic. The user can select at least two statistics to generate a custom projection, which will then be used to display the players in the scatter plot. This feature is particularly useful for advanced users who wish to fine-tune their search criteria.

Once the desired statistics are toggled on, the user can click the "Apply Configuration" button. Then, the user must wait for the new custom projection to be calculated. Upon completion, the window will close. By selecting "custom projection" in the projection dropdown menu, these values will be visualized in the plot.

5 Machine Learning

5.1 Developing a Scoring Metric

One key aspect of our tool is the ability to filter players using various scores derived from our database, each ranging between 0 and 100. A higher score signifies greater proficiency in a particular category. As detailed in section 4.2, the radar chart facilitates direct player comparisons, leveraging these scores. Integrating these scores into our system presented significant challenges and necessitated some design decisions. The primary challenge stemmed from our unlabeled data, rendering supervised learning unsuitable for this task. Another major challenge was determining the number of distinct categories for scoring and identifying these categories.

Initially, we divided our feature set into offensive, defensive, and miscellaneous features. We applied the KMeans++[1] clustering algorithm to the offensive features, using the silhouette score[3] to evaluate clustering quality and determine the optimal number of clusters. Three clusters yielded the best results. For feature selection, we employed a sequential feature selection (SFS) algorithm, which iteratively greedily added features to the set by maximizing the silhouette score. We modified the standard SFS algorithm because the original version excluded some desired features. Thus, we

established a set of "base features" for the SFS algorithm, including 'true shooting percentage,' 'effective field goal percentage,' and 'assists.' Additionally, we prevented SFS from selecting features already captured by the base features.

The rationale for using clustering was to identify three distinct groups within the offensive feature sub-space. Analyzing these clusters inspired the creation of three offensive scoring categories. We incorporated cluster labels into the pre-normalization dataset and computed statistics for each cluster, such as mean, variance, and various percentiles. Through manual analysis, we identified approximately five features per cluster that were significantly different from those in other clusters. Our basketball knowledge ensured the selected features were coherent, and expert opinions could have further refined our model. We named the categories "Playing in the Paint," "Perimeter Offense," and "Driving/Penetrating Offense". Details can be found in our dashboard.

Now, to generate scores, we fitted three multivariate Gaussian distributions to the selected feature groups and applied the resulting cumulative distribution functions (CDFs) to each player, yielding preliminary scores. The intuition behind this is that a higher CDF indicated higher feature values for a player. However, these scores were not uniformly distributed between 0 and 100, leading to sensitivity issues with filtering near the boundaries, as most players would be around the middle area. To address this, we ranked players from 1 to #players in each offensive category and scaled their rankings to fall between 0 and 100, achieving uniform score distribution.

The clustering-based approach proved ineffective for defensive and miscellaneous features, which is why we decided to adopt a different strategy. We utilized the "Defensive Performance Indicator" (DPI) from [4] to formulate a simple scoring method for defensive capabilities. The DPI, which accurately predicted the NBA's defensive player of the year, includes three features we didn't have access to. Consequently, we simply excluded these features from our defensive scoring formula.

For the fifth and final category, we ranked players by their rebounding performance, a critical metric in player analysis. We applied the same ranking and scaling method used for offensive features to both rebounding and defensive scores, ensuring uniformly distributed scores. That is, we first ranked the players from 1 to #players (each on rebounding stats and DPI), then scaled and shifted their rankings to fall between 0 and 100. Lastly, it is noteworthy that the clustering-inspired approach revealed groups corresponding to traditional basketball positions: "Perimeter Offense" aligns with point guards, "Playing in the Paint" with centers, and "Driving/Penetrating Offense" with shooting guards or forwards.

5.2 Projections

As a second approach to including machine learning methods in our project, we explored different unsupervised learning strategies to help users interpret the inherently high-dimensional data. Ultimately, we decided to use t-Distributed Stochastic Neighbor Embedding (t-SNE), a non-linear projection method that reduces high-dimensional data to two dimensions. This allows users to select specific statistics they are interested in, and the resulting projection based on these features helps them identify players with similar characteristics. This feature is particularly useful for scouting players, such as when a team needs to find a replacement for a departing player with a comparable play style.

t-SNE is a robust dimensionality reduction technique designed to create a low-dimensional (in this case, 2D) representation of high-dimensional data preserving the relative distances between points. It models the similarity of two points as conditional probabilities and optimizes the low-dimensional coordinates to minimize the Kullback-Leibler divergence between the high-dimensional and low-dimensional distributions. While there are several hyperparameters to tune, we found that the default settings in scikit-learn[2], as recommended by the original t-SNE paper[5], work well for our application.

However, working with dimensionality reduction presents some challenges. For instance, the axes in the 2D t-SNE plot do not correspond to meaningful directions; there is no specific direction where players improve. This can be confusing for users unfamiliar with this type of representation. Additionally, absolute distances in the plot are not informative; the focus is on the relative distances between players. To prevent misinterpretations and ensure users draw accurate conclusions from the visualization, we made these points explicit to them.

5.3 Explainability

Explainability for the ML methods is a critical aspect of our tool, ensuring that the insights generated are transparent and actionable for scouts. We provide explanations for the projections used, which are accessible via tooltips and help icons within the interface. This feature helps users to understand what currently is being represented on the dashboard, allowing them to make informed decisions based on the data.

6 Conclusion

Our initial goal was to create a dashboard that could help a scout find new players through a visual, interactive, and ML-supported approach. We believe we achieved this goal. Key areas for potential improvement include the refinement of the scoring metric with input from domain experts. Additionally, our current models are trained on a static dataset, which presents limitations. Ideally, the model should dynamically adjust as new data is added, such as at the end of each season. These are just two of many possible improvements, and we acknowledge there is always room for further development. Nevertheless, we believe that our tool, in its current state, offers valuable support for scouting basketball players and can be effectively used in real-world scenarios.

References

[1] D. Arthur, S. Vassilvitskii, et al. k-means++: The advantages of careful seeding. In *Soda*, vol. 7, pp. 1027–1035, 2007. 3

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. 4

[3] P. J. Rousseeuw. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of computational and applied mathematics*, 20:53–65, 1987. 3

[4] V. Sarlis and C. Tjortjis. Sports analytics—evaluation of basketball players and team performance. *Information Systems*, 93:101562, 2020. 4

[5] L. van der Maaten and G. Hinton. Visualizing high-dimensional data using t-sne. *Journal of Machine Learning Research*, 9(nov):2579–2605, 2008. Pagination: 27. 4